openioc_scan

Takahiro Haruyama (@cci_forensics)
Internet Initiative Japan Inc.

SECURE 2015

Who am I?

- Forensic Investigator & Malware Analyst at Internet Initiative Japan Inc.
  - IIR: Internet Infrastructure Review
    - http://www.iij.ad.jp/en/company/development/iir/index.html
- Presentations and Hands-on classes
  - Black Hat Briefings USA/Europe/Asia, SANS DFIR Summit, DFRWS EU, CEIC, FIRST TC, etc...
  - Keywords
    - anti forensics, memory forensics, IOC, targeted attack, PlugX
- Blog
  - http://takahiroharuyama.github.io/
    - plugins/scripts for Volatility Framework, IDA Pro, Immunity Debugger and EnCase

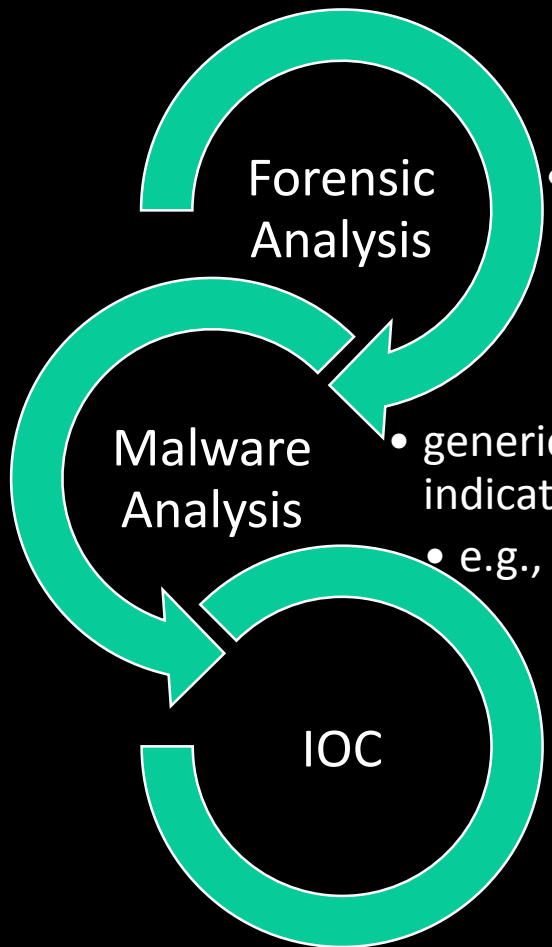
Agenda

- Introduction
- Tips about Memory Forensic IOCs
- openioc_scan vs. Equation Group
- Remote Malware Triage
- Wrap-up

Introduction

IOC (Indicator Of Compromise)

- A piece of information that can be used to search for or identify potentially compromised systems [1]



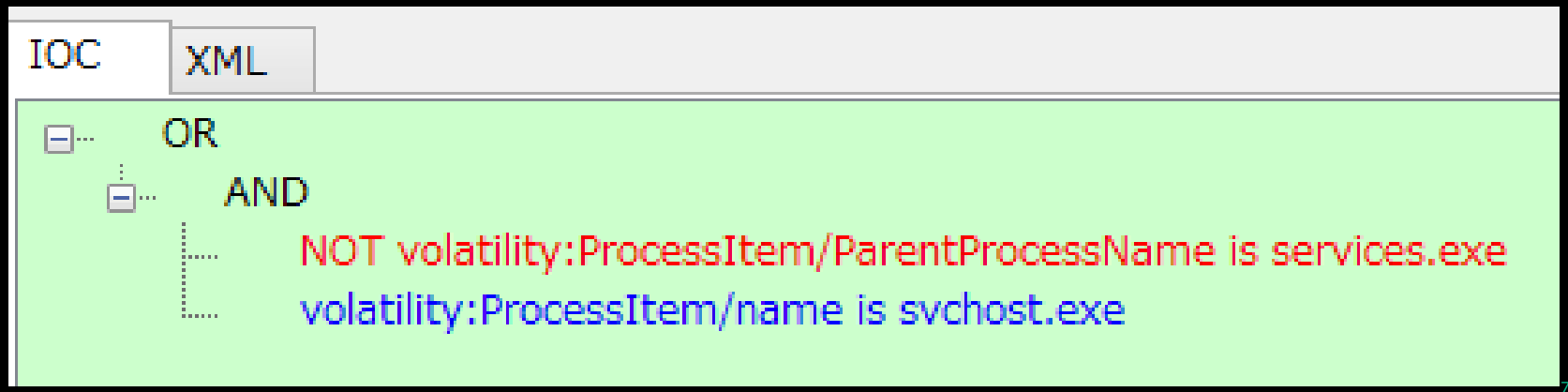
- specific indicators
  - e.g., URL, file hash
- generic (function-based) indicators
  - e.g., used API, binary code
- define & improve
- scan on live system, disk image, memory image

openioc_scan

- Memory forensic IOC scanner implemented as a plugin of Volatility Framework [2]
  - scans IOCs written in OpenIOC 1.1 format
    - case sensitiveness, regular expression (“matches” condition), parameters [3]
  - supports only Windows (Vista or later)
  - 3 python packages required
    - lxml [4]
    - ioc_writer [5]
    - colorma [6]

The 1st IOC example

- Generating IOC for openioc_scan
  - PyIOCe [7] made by Sean Gillespie
    - The latest “terms” and “parameters” of volatility should be imported [8][9]
- The following IOC means
  - malicious *svchost* process NOT created from *services* process



The 1st IOC example (Cont.)

- openioc_scan options
  - -i IOC_DIR, --ioc_dir=IOC_DIR
    - location of IOCs directory (required)
  - -s, --show
    - display mode (not scan)
  - -t, --test
    - test mode (useful for debugging IOC)

PlugX detected

```
c-dfe9c1ca240c
**********************************************************************
IOC matched (by logic)! short_desc="rogue svchost" id=a50223b5-b213-43e9-beac-df
e9c1ca240c
logic (matched item is magenta-colored):
  (
    >>> Not ProcessItem/ParentProcessName is services.exe
    and
    >>> ProcessItem/name is svchost.exe
  )
Note: ProcessItem was evaluated only in svchost.exe (Pid=2204)
**********************************************************************
INFO    : volatility.plugins.openioc_scan: => elapsed scan total: about 5.456000
```

Supported 35 IOC Terms

- Check the blog entry for more information [8]

Term Category	Term Examples
ProcessItem	name, command line, parent name, DLL path, process/DLL DKOM detection, code injection detection, imported/dynamic generated API table, string, handle name, network connection, IAT/EAT/inline hooked API name, enabled privilege name
RegistryItem	metadata of executables cached by OS (ShimCache)
ServiceItem	service name/description/command line
DriverItem	name, imported/dynamic generated API table, string, hooked IRP function table, callback function type, timer function detection
HookItem	hooked SSDT entry
FileItem	filename/size/path based on carved MFT entry

Tips about Memory Forensic IOCs

Specific IOC vs. Generic IOC

- 2 IOC types in memory forensics
  - Specific IOC
    - e.g., unique URL, string, binary data/code sequence
  - Generic IOC
    - e.g., suspicious process tree, code injection sign, API function name used/hooked by malware
- Generic ones recommended in less urgent situations

	specific IOC	**generic IOC**
advantage	easy to define (low false positive rate)	detect unknown malware with similar traits
weakness	detect the malware only	hard to define (high false positive rate)

Keep It Simple and Short

- Good IOCs should be simple
- e.g., generic code injection detection
  - Injected codes
    - often hook APIs (e.g., HttpSendRequest*, PR_Write)
    - cannot be resolved from module linked lists (e.g., InLoadOrderModuleList)

Andromeda

```
**********************************************************
IOC matched (by logic)! short_desc="unknown hooking module name"
6
logic (matched item is magenta-colored):
  >>> ProcessItem/Hooked/API/HookingModuleName contains unknown
Note: ProcessItem was evaluated only in explorer.exe (Pid=1108)
**********************************************************
```

Tinba

```
**********************************************************
IOC matched (by logic)! short_desc="unknown hooking module name"
6
logic (matched item is magenta-colored):
  >>> ProcessItem/Hooked/API/HookingModuleName contains unknown
Note: ProcessItem was evaluated only in firefox.exe (Pid=3176)
**********************************************************
```

Keep It Simple and Short #2

- e.g., process hollowing [10] detection
  - Process hollowing technique hides malicious codes into legitimate processes
    1. create a new process with suspended state
    2. free the process code and replace with malware
    3. change the process context and resume it
  - Merit
    - Malware runs as the process
      - The file path seems to be legitimate
      - Malware takes privileges of the process (e.g., Firewall exception)

```
*****************************************************************
lsass.exe pid:    868
Command line : "C:¥WINDOWS¥¥system32¥¥lsass.exe"
Service Pack 3

Base              Size  LoadCount Path
0x01000000      0x6000     0xffff C:¥WINDOWS¥system32¥lsass.exe
0x7c900000     0xaf000     0xffff C:¥WINDOWS¥system32¥ntdll.dll
0x7e800000      0xf6000    0xffff C:¥WINDOWS¥system32¥kernel32.dll
```

hollowed process path from PEB

Keep It Simple and Short #2 (Cont.)

- e.g., process hollowing [10] detection
  - Detection
    - Freeing the process code changes VAD (Virtual Address descriptor) characteristics [11]

Pid	Process	Base	InLoad	InInit	InMem	MappedPath
868	lsass.exe	0x00080000	False	False	False	
868	lsass.exe	0x7c900000	True	True	True	¥WINDOWS¥s
868	lsass.exe	0x77e70000	True	True	True	¥WINDOWS¥s
868	lsass.exe	0x7c800000	True	True	True	¥WINDOWS¥s
868	lsass.exe	0x77fe0000	True	True	True	¥WINDOWS¥system32¥secur32.dl
868	lsass.exe	0x7e410000	True	True	True	¥WINDOWS¥system32¥user32.dll
868	lsass.exe	0x01000000	True	False	True	
868	lsass.exe	0x77f10000	True	True	True	¥WINDOWS¥system32¥gdi32.dll

Path from FileObject in VAD is null

Stuxnet

```
*****************************************************************
IOC matched (by logic)! short_desc="Process/module hollowing" id=
logic (matched item is magenta-colored):
  >>> ProcessItem/DllPath is
Note: ProcessItem was evaluated only in lsass.exe (Pid=868)
*****************************************************************
```

Focus on Functions

- IOCs based on malware functions become generic automatically
- e.g., detecting malware hiding data in NTFS $EA
  - NTFS $EA (Extended Attribute) is provided for backward compatibility with OS/2 applications
  - Two APIs: Zw(Nt)QueryEaFile and Zw(Nt)SetEaFile
    - ZeroAccess (user mode) and Regin (kernel mode) use the APIs
  - openioc_scan checks not only IAT but also dynamically-generated API tables

ZeroAccess

```
**********************************************************************
IOC matched (by logic)! short_desc="using NTFS $EA (process)" id=b61f88d5-9453-4
69b-94cd-c5ef59c972db
logic (matched item is magenta-colored):
  >>> ProcessItem/SectionList/MemorySection/PEInfo/ImportedModules/Module/Import
edFunctions/string contains QueryEaFile
  or
  >>> ProcessItem/SectionList/MemorySection/PEInfo/ImportedModules/Module/Import
edFunctions/string contains SetEaFile
Note: ProcessItem was evaluated only in services.exe (Pid=556)
**********************************************************************
```

Use Parameter

- OpenIOC 1.1 supports “Parameter” (metadata for each IOC term)
- Detail Parameter: display not only matched substring but also total one
- e.g., unusual path detection
  - Malware often runs from abnormal locations

Dridex

```
matched IOC term detail: ¥Users¥[redacted]¥AppData¥Local¥edg7A89.exe
**********[redacted]
IOC matched (by logic)! short_desc="suspicious paths (running)" id=

logic (matched item is magenta-colored):
  (
    ProcessItem/cmdLine contains ¥ProgramData (detail=on;)
    or
    ProcessItem/cmdLine contains ¥$Recycle.Bin (detail=on;)

   or
   >>> ProcessItem/cmdLine matches ¥¥Users¥¥.*¥¥AppData (detail=on;)
  )
  or
```

parameter: detail=on

Use Parameter #2

- By adding Score Parameter, openioc_scan additionally evaluates IOCs based on the total of its integer values (>=100)
- e.g., information-stealing malware detection
  - Hooked HttpSendRequest APIs and unknown hooking module name (due to code injection)

```
*****************************************************************
IOC matched (by score)! short_desc="Information-Stealing Malware" id=2823537b-8c9a-
54
logic (matched item is magenta-colored):
  (
    >>> ProcessItem/Hooked/API/FunctionName contains HttpSendRequestA (score=40;)
    and
    >>> ProcessItem/Hooked/API/FunctionName contains HttpSendRequestW (score=40;)
    and
    ProcessItem/Hooked/API/FunctionName contains HttpSendRequestExA (score=40;)
    and
    ProcessItem/Hooked/API/FunctionName contains HttpSendRequestExW (score=40;)
    and
    >>> ProcessItem/Hooked/API/HookingModuleName contains unknown (score=20;)
  )
Note: ProcessItem was evaluated only in explorer.exe (Pid=1796)
*****************************************************************
```

Consider Performance

- Scanning all IOCs at one time is STUPID
  - prioritize fast IOC terms over slow ones
    - fast: e.g., process name/path, network connection
    - slow: e.g., string, binary sequence, used/hooked API name
- Avoid combination of multiple IOC term categories
  - Especially, ProcessItem and DriverItem are heavily iterated
  - Define them separately, or limit the scope by adding –p (process) / -m (driver) options
- Information extracted by openioc_scan is cached
  - The 2nd time scan for the same term gets the result in less time
    - searched in cached SQLite database
  - Binary sequences, however, are NOT cached
    - instead, searched in individual process/driver memory dumps

openioc_scan vs. Equation Group

What’s Equation Group?

- Threat actor introduced by Kaspersky in Feb. 2015 [12]
  - used malware sets including EquationLaser, EquationDrug, DoubleFantasy, TripleFantasy, Fanny, GrayFish
- HDD firmware reprogramming module (nls_933w.dll) is loaded on EquationDrug and GrayFish
  - It generates hidden data area in HDD
  - but still exists in RAM for providing API into the area!

nls_933w.dll Behavior [13]

- nls_933w.dll creates win32m.sys then communicate with it using DeviceIoControl API
  - 6 IoControlCodes used
    - The code “0x870021D0” is used for read/write requests of ATA device registers
      - A kernel timer function handles the requests
    - The data to write is embedded in the dll
      - A combination of 6-byte data structure

```
; enum win32m_ControlCode
IOCTL_WIN32M_GET_VERSION  = 870021C0h
IOCTL_WIN32M_INIT_IRQL_DPC_SPINLOCK  = 870021C4h ; set IO handlers
IOCTL_WIN32M_CLEAR_IRQL_DPC_SPINLOCK  = 870021C8h ; unset IO handlers
IOCTL_WIN32M_GET_LP_INFO  = 870021CCh
IOCTL_WIN32M_QUEUE_IO_REQUEST  = 870021D0h
IOCTL_WIN32M_SET_PARAMETERS  = 870021D4h
```

read/write of ATA device registers

```
g_ATA_0xEC      dd 0            ; field_0_flag
                                ; DATA XREF: fn_ctr_Obj_ATA_0xEC+7↑o
                                ; 0=write, other=read ; length = 0xC
                db 6            ; field_4_reg_offset
                                ; 0=Data(R/W), 1=Error(R) or Features(W
                db 0            ; field_5_data
                dd 0            ; field_0_flag
                                ; 0=write, other=read
                db 7            ; field_4_reg_offset
                                ; 0=Data(R/W), 1=Error(R) or Features(W
                db 0ECh         ; field_5_data
```

“IDENTIFY_DEVICE” command

nls_933w.dll IOCs

- Specific IOCs
  - dll: IoControlCodes and 6-byte data structure
  - driver: IoControlCodes and a binary code sequence parsing the structure
- Generic IOC
  - driver: APIs for ATA device IO and kernel timer

```
*****************************************************************
IOC matched (by logic)! short_desc="EquationDrug HDD/SSD firmware operation (kernel,generic)" id=e2bc
db-dbfd-45f8-a81d-24314516d0c6
logic (matched item is magenta-colored):
  (
    >>> DriverItem/PEInfo/ImportedModules/Module/ImportedFunctions/string contains WRITE_PORT_ULONG
    and
    >>> DriverItem/PEInfo/ImportedModules/Module/ImportedFunctions/string contains WRITE_PORT_USHORT
    and
    >>> DriverItem/PEInfo/ImportedModules/Module/ImportedFunctions/string contains WRITE_PORT_BUFFER

    and
    >>> DriverItem/TimerRoutineIncluded is true
  )
Note: DriverItem was evaluated only in win32m.sys (base=0xf9d29000)
*****************************************************************
```

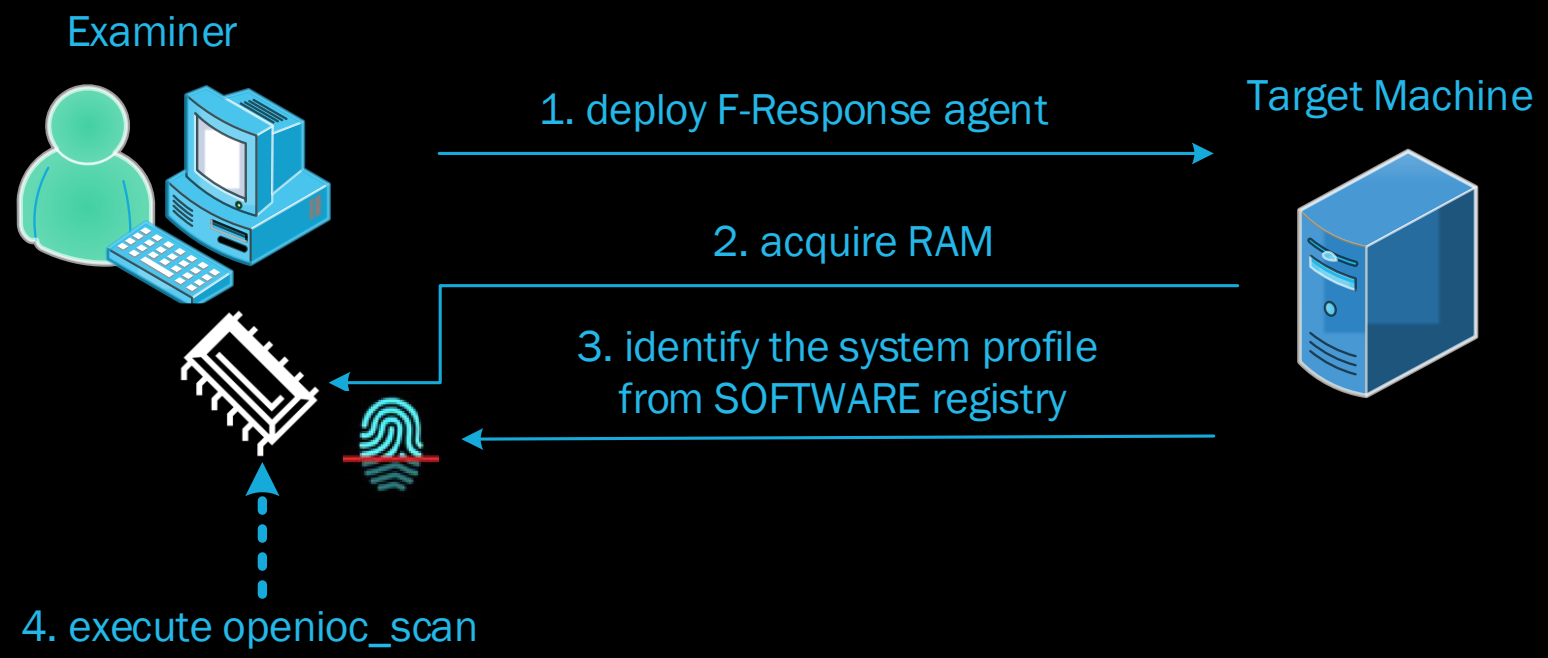
Remote Malware Triage

onigiri.py - Remote Malware Triage Automation [14]

- openioc_scan + F-Response [15]

= remote malware triage!

- F-Response provides read-only access to the full physical disk(s) of any networked computer
  - plus the physical memory (RAM) of most Microsoft Windows systems
- We can automate RAM acquisition from remote machine and IOC scan using F-Response COM API



Usage

- It depends on the F-Response edition
  - Enterprise provides COM APIs for Enterprise Management Console (EMC)
    - We can fully automate including the remote agent deployment
  - Consultant and Consultant+Covert provide APIs for Consultant Connector (CC)
    - We must deploy the agent manually
      - Consultant: export it to USB thumb drive then run on the target PC
      - Consultant+Covert: use Consultant+Covert console (“Direct Connect” menu)

Usage (Cont.)

- Some preparations needed (3rd party tools, settings,..)
- [64bit python path] onigiri.py [actions] [options] [output] [edition] [arguments for edition]
- actions
  - -r: RAM acquisition
  - -f: file acquisition by file categories (later)
  - -s: IOC scan with RAM and SOFTWARE registry acquisition
- options
  - IOC folder path, tool paths, admin credentials,..
- edition
  - emc or cc

DEMO (EMC)

```
C:¥tmp>"C:¥Python27¥python.exe" onigiri.py -s -i C:¥ioc_mine C:¥tmp¥onigiri-out emc 192.168.91.130,192.168.91.138 hoge forensics -d HOGE
main:INFO: onigiri.py - remote malware triage
main:INFO: ########### STEP1: RAM/files Acquisition ############
FRESemc:INFO: Directly-deploying agent into the victim machine: 192.168.91.130 (Status=1)...
FRESemc:INFO: Agent ready: 192.168.91.130 (Platform=0, Status=3)
FRESemc:INFO: Starting RAM Acquisition
FRESemc:INFO: Physical Memory found: iqn.2008-02.com.f-response.vistaenterprise:pmem (DiskType=3)
FRESemc:INFO: acquiring mapped physical memory (¥¥.¥PhysicalDrive1)...
FRESemc:INFO: RAM image saved: C:¥tmp¥onigiri-out¥192.168.91.130¥pmem.dd4.001
FRESemc:INFO: Starting File Acquisition
FRESemc:INFO: file saved: C:¥tmp¥onigiri-out¥192.168.91.130¥vol-C¥Windows¥System32¥config¥SOFTWARE
FRESemc:INFO: stopping/uninstalling agent...
FRESemc:INFO: Directly-deploying agent into the victim machine: 192.168.91.138 (Status=1)...
FRESemc:INFO: Agent ready: 192.168.91.138 (Platform=0, Status=3)
FRESemc:INFO: Starting RAM Acquisition
FRESemc:INFO: Physical Memory found: iqn.2008-02.com.f-response.win2008-dc:pmem (DiskType=3)
FRESemc:INFO: acquiring mapped physical memory (¥¥.¥PhysicalDrive1)...
FRESemc:INFO: RAM image saved: C:¥tmp¥onigiri-out¥192.168.91.138¥pmem.dd4.001
FRESemc:INFO: Starting File Acquisition
FRESemc:INFO: file saved: C:¥tmp¥onigiri-out¥192.168.91.138¥vol-C¥Windows¥System32¥config¥SOFTWARE
FRESemc:INFO: stopping/uninstalling agent...
main:INFO: calculating SHA1 hashes of the acquired files...
main:INFO: ########### STEP2: Scanning IOCs in RAM ############
IOC scan target: 192.168.91.130
Profile identified from SOFTWARE registry: VistaSP2x86
running openioc_scan...
C:¥Python27x86¥python.exe C:¥volatility¥vol.py openioc_scan --profile=VistaSP2x86 --ioc_dir=C:¥ioc_mine -f C:¥tmp¥onigiri-out¥192.168.91.
¥pmem.dd4.001 -e
Volatility Foundation Volatility Framework 2.4
INFO    : volatility.plugins.malware.openioc_scan: loading IOCs from: C:¥ioc_mine
INFO    : volatility.plugins.malware.openioc_scan: Parsed [1] IOCs
INFO    : volatility.plugins.malware.openioc_scan: Clearing the tables... (old DB or erase flag)
INFO    : volatility.plugins.malware.openioc_scan: Making new DB tables...
INFO    : volatility.plugins.malware.openioc_scan: [time-consuming task] extracting all processes including hidden/dead ones...
INFO    : volatility.plugins.malware.openioc_scan: 56 processes found
INFO    : volatility.plugins.malware.openioc_scan: Scanning iocid=bb040abe-2e3b-4571-8834-c876abcd9abf
matched IOC term detail: f-response-ent.exe
*************************************************************
IOC matched (by logic)! short_desc="process commandline" id=bb040abe-2e3b-4571-8834-c876abcd9abf
logic (matched item is magenta-colored):
  >>> ProcessItem/cmdLine contains response (detail=on;)
Note: ProcessItem was evaluated only in f-response-ent. (Pid=2824)
*************************************************************
INFO    : volatility.plugins.malware.openioc_scan: => elapsed scan total: about 15.1359996796 s
IOC scan target: 192.168.91.138
Profile identified from SOFTWARE registry: Win2008R2SP1x64
```

"C:¥Python27¥python.exe" onigiri.py -s -i C:¥ioc_mine C:¥tmp¥onigiri-out emc 10.xxx.xxx.xxx

Notes

- onigiri supports not only a RAM acquisition but also an acquisition including files with unallocated (deleted) status
  - sysreg,userreg,mft,prefetch,evtx,amcache,journal
  - F-Response Flexdisk API used
- RAM acquisition may fail on Win 8.1 x64
  - F-Response bug (not fixed)
  - Use an alternate acquisition option using DumpIt 2.x (paid version) [16]
    - But DumpIt also may cause BSOD when using remote acquisition option ☹ (not fixed)
- If any other errors, check “Trouble Shooting” in README on GitHub

Wrap-up

Wrap-up

- Are you still detecting malware based on only hash values?
  - openioc_scan enables to detect faster and deeper than disk-based traditional IOCs
- Tools are available on GitHub
  - openioc_scan and generic/specific IOC examples
    - https://github.com/TakahiroHaruyama/openioc_scan
  - onigiri (remote malware triage script)
    - https://github.com/TakahiroHaruyama/onigiri

Reference

- [1] Sharing Indicators of Compromise: An Overview of Standards and Formats
  - https://www.rsaconference.com/writable/presentations/file_upload/dsp-w25a.pdf
- [2] volatilityfoundation/volatility
  - https://github.com/volatilityfoundation/volatility
- [3] mandiant/OpenIOC_1.1
  - https://github.com/mandiant/OpenIOC_1.1
- [4] lxml 3.2.1 : Python Package Index
  - https://pypi.python.org/pypi/lxml/3.2.1
- [5] mandiant/ioc_writer
  - https://github.com/mandiant/ioc_writer
- [6] colorama 0.3.3 : Python Package Index
  - https://pypi.python.org/pypi/colorama
- [7] yahoo/PyIOCe
  - https://github.com/yahoo/PyIOCe
- [8] Fast Malware Triage Using Openioc_scan Volatility Plugin
  - http://takahiroharuyama.github.io/blog/2014/08/15/fast-malware-triage-using-openioc-scan-volatility-plugin/

Reference (Cont.)

- [9] OpenIOC Parameters Used by Openioc_scan
  - http://takahiroharuyama.github.io/blog/2014/10/24/openioc-parameters-used-by-openioc-scan/
- [10] Analyzing Malware Hollow Processes
  - https://www.trustwave.com/Resources/SpiderLabs-Blog/Analyzing-Malware-Hollow-Processes/
- [11] “The Art of Memory Forensics” p.261
  - http://as.wiley.com/WileyCDA/WileyTitle/productCd-1118825098.html
- [12] “Equation group: questions and answers”
  - https://securelist.com/files/2015/02/Equation_group_questions_and_answers.pdf
- [13] Internet Infrastructure Review (IIR) Vol.27
  - http://www.iij.ad.jp/en/company/development/iir/027.html
- [14] Remote Malware Triage Automation
  - http://takahiroharuyama.github.io/blog/2015/07/03/remote-malware-triage-automation/
- [15] F-Response
  - https://www.f-response.com/
- [16] MoonSols Windows Memory Toolkit
  - http://www.moonsols.com/windows-memory-toolkit/